

CAPSTONE RESEARCH PROPOSAL

Intelligent LiDAR-Based Aircraft Stall Margin Prediction Using Artificial Intelligence and Sensor Fusion Under Atmospheric Turbulence

A predictive, forward-looking alternative to reactive stall-warning systems

Prepared by: Nistor Darius
Date: July 2026

Research prototype for academic purposes only — not a certified avionics system

Abstract

Loss of control in flight, frequently precipitated by aerodynamic stall, remains one of the most persistent causes of fatal aviation accidents during takeoff, approach, and encounters with atmospheric turbulence. Current stall protection depends on angle-of-attack vanes, pitot-static systems, and inertial measurements that characterize the aircraft's state only after airflow separation has begun or is imminent — leaving pilots with a narrow and reactive window to respond.

This proposal outlines a research prototype for a predictive stall-margin estimation system that senses disturbed air ahead of the aircraft using forward-looking LiDAR, fuses this data with conventional flight parameters, and applies machine learning to estimate the aircraft's remaining stall margin several seconds before the critical angle of attack is reached. The system is intended as a decision-support and research tool rather than a certified flight-critical component.

The proposed work spans sensor integration, embedded data acquisition, multi-sensor fusion, supervised machine learning, simulator-based validation using FlightGear or X-Plane, and a cockpit-style visualization interface. The deliverables include a working hardware/software prototype, a labeled dataset generated in simulation, trained classification and regression models for stall-risk estimation, and a quantitative evaluation of prediction accuracy and lead time. The project is scoped to be achievable as a two-semester undergraduate or master's capstone, with clearly separated stretch goals for teams wanting to pursue conference-paper-quality results.

Table of Contents

Abstract

Chapter 1. Introduction

- 1.1 Background and Motivation
- 1.2 Problem Statement
- 1.3 Research Objectives
- 1.4 Research Questions
- 1.5 Expected Contributions
- 1.6 Scope and Limitations

Chapter 2. Literature Review

- 2.1 Aerodynamic Fundamentals and Stall
- 2.2 Atmospheric Turbulence and Gust Encounters
- 2.3 Existing Onboard Stall Protection Systems
- 2.4 Remote Airflow Sensing: LiDAR and Related Technologies
- 2.5 Sensor Fusion for Flight State Estimation
- 2.6 Machine Learning in Aviation Safety
- 2.7 Research Gap

Chapter 3. System Architecture and Design

- 3.1 Concept of Operations
- 3.2 High-Level Architecture
- 3.3 Sensor Placement
- 3.4 Data Flow
- 3.5 Communication and Interfaces

Chapter 4. Hardware Design

- 4.1 Component Selection Rationale
- 4.2 Sensor Placement Diagram
- 4.3 Power and Integration Considerations
- 4.4 Honest Limitations of the Sensing Approach

Chapter 5. Software Design

- 5.1 Software Architecture
- 5.2 Technology Stack
- 5.3 Module Responsibilities

Chapter 6. Mathematical Framework

- 6.1 Lift and Drag
- 6.2 Effective Angle of Attack Under Gust
- 6.3 Stall Margin

6.4 Stall Probability as a Learned Function

6.5 Evaluation Metrics

Chapter 7. Machine Learning Methodology

7.1 Problem Framing

7.2 Candidate Features

7.3 Target Labels

7.4 Candidate Models

7.5 Training and Validation Protocol

7.6 Explainability

Chapter 8. Reference Implementation

8.1 Deterministic Baseline (Rule-Based)

8.2 Illustrative Machine Learning Prototype

8.3 Software Modules to Implement

Chapter 9. Simulation and Test Plan

9.1 Simulation Environment

9.2 Test Scenarios

9.3 Metrics Collected Per Run

9.4 Hardware-in-the-Loop (Optional Stretch Goal)

Chapter 10. Expected Results and Evaluation

10.1 Success Criteria

10.2 Reporting Format

10.3 Threats to Validity

Chapter 11. Future Work

Chapter 12. Conclusion

Appendix A — Indicative Project Timeline

Appendix B — Reference Sources to Consult

Chapter 1. Introduction

1.1 Background and Motivation

Stall occurs when the angle of attack (AoA) between the wing and the oncoming airflow exceeds a critical value, causing boundary-layer separation and a rapid loss of lift. Stall-related loss-of-control accidents continue to appear disproportionately in accident statistics published by aviation safety authorities, particularly in general aviation and during low-altitude phases of flight such as takeoff, approach, and go-around, where recovery altitude is limited.

Existing stall protection — angle-of-attack indicators, stick shakers, stick pushers, and envelope-protection laws such as Airbus's Alpha Floor or Boeing's Stall Management Yaw Damper — are fundamentally reactive: they detect that the aircraft's own state (measured AoA, airspeed decay, load factor) is already approaching a stall condition. None of these systems directly measure the disturbed air ahead of the wing, so a sudden gust or wind-shear encounter is only visible to the aircraft after it has already altered the local flow field.

Advances in solid-state and mechanical LiDAR, combined with low-cost embedded AI compute (e.g., edge GPUs), make it feasible for a research project to explore whether forward-sensing of air disturbances can extend the effective warning time before stall onset — shifting stall protection from reactive to anticipatory.

1.2 Problem Statement

Current onboard stall-warning systems provide little to no lead time in gust- or turbulence-induced stall encounters because they rely exclusively on sensors that characterize the aircraft's own motion rather than the approaching airflow. There is no low-cost, prototype-accessible system that fuses forward-looking remote-sensing data with aircraft state to produce an early, quantified stall-margin estimate.

1.3 Research Objectives

- Design and integrate a forward-looking LiDAR sensing arrangement capable of detecting airflow-relevant disturbances ahead of a wing section in a laboratory or simulated setting.
- Develop a sensor-fusion pipeline that combines LiDAR-derived gust/wind estimates with aircraft telemetry (AoA, airspeed, attitude, altitude).
- Train and evaluate machine learning models that estimate effective angle of attack, stall margin, and stall probability several seconds ahead of the event.
- Validate the system in a flight-simulator environment across a range of turbulence and gust scenarios.
- Design a cockpit-style visualization that communicates stall margin and recommended pilot action in an intuitive form.

1.4 Research Questions

- Can forward-looking LiDAR provide a measurable lead-time advantage over conventional AoA-based stall warning in a simulated environment?
- What sensor-fusion and feature-engineering approach best converts raw point-cloud and telemetry data into a reliable stall-margin estimate?
- Which class of machine learning model (tree ensemble, feed-forward network, or sequence model such as an LSTM) offers the best accuracy-versus-latency trade-off for this task on embedded hardware?

1.5 Expected Contributions

- A working, documented hardware/software prototype integrating LiDAR, IMU, and an embedded AI compute module.
- A simulator-generated, labeled dataset relating gust/turbulence conditions to stall margin, suitable for reuse in follow-on research.
- A quantitative comparison of predictive lead time versus conventional AoA-threshold warning.
- A candidate architecture and set of design considerations for future certification-track development (discussed, not achieved, in this project).

1.6 Scope and Limitations

This project is a research and educational prototype. It is not a certified avionics system and is not intended for installation on a certified aircraft. All flight validation is performed in simulation (FlightGear or X-Plane) rather than on a real airframe. Hardware-in-the-loop testing, if pursued, should be limited to ground-based rigs or, where institutional safety review permits, a non-certified experimental airframe under appropriate supervision. Certification standards such as DO-178C (software), DO-254 (hardware), and ARINC 429 (data bus) are referenced in Chapter 11 as future considerations rather than requirements of this capstone.

Chapter 2. Literature Review

2.1 Aerodynamic Fundamentals and Stall

A wing generates lift by accelerating air over its upper and lower surfaces to create a pressure differential. As angle of attack increases, lift coefficient increases approximately linearly until the boundary layer on the upper surface begins to separate. Beyond the critical angle of attack, separation dominates, lift coefficient falls, and drag rises sharply — the aerodynamic stall. Because stall is governed by angle of attack rather than airspeed directly, an aircraft can stall at any airspeed or attitude if AoA becomes excessive, which is why AoA — not airspeed — is the fundamental parameter of interest for this project.

2.2 Atmospheric Turbulence and Gust Encounters

Gusts and turbulence — from convective activity, mechanical turbulence over terrain, wind shear, and wake vortices from other aircraft — instantaneously change the local relative wind seen by the wing, producing a transient increase in effective angle of attack that is not reflected in the aircraft's pre-gust AoA reading. Classical gust-load models (e.g., the discrete '1-cosine' gust and continuous Dryden/von Kármán turbulence spectra used throughout aerospace guidance and simulation standards) provide a well-established basis for generating realistic synthetic turbulence for training and testing the proposed system.

2.3 Existing Onboard Stall Protection Systems

Modern transport aircraft use a mix of AoA vanes, stick shakers/pushers, and flight-envelope protection laws. Airbus fly-by-wire aircraft implement an 'Alpha Floor' protection that commands go-around thrust when AoA approaches a protected limit; Boeing aircraft have used a Stall Management Yaw Damper and related AoA-based alerting. General aviation aircraft increasingly retrofit standalone AoA indicators following FAA guidance intended to reduce loss-of-control accidents. In every case, the underlying measurement is the aircraft's own current state, not the air ahead of it — the gap this project targets.

2.4 Remote Airflow Sensing: LiDAR and Related Technologies

Airborne Doppler LiDAR and Doppler wind LiDAR have been explored by research organizations for wake-vortex detection, wind-shear alerting, and clear-air turbulence research, typically using specialized, aircraft-grade coherent LiDAR at significant cost. Lower-cost time-of-flight and solid-state LiDAR units, originally developed for automotive and robotics applications, cannot directly measure wind velocity the way a Doppler LiDAR can; in this project they are used primarily as a proxy sensor — detecting particulate/aerosol-scale returns and relative motion cues — with the explicit understanding that a full Doppler-wind capability is a stretch goal rather than a baseline requirement. This distinction is discussed further in Chapter 4.

2.5 Sensor Fusion for Flight State Estimation

Sensor fusion techniques — Kalman filtering, complementary filtering, and more recently learned fusion architectures — are the standard approach for combining noisy, asynchronous sensors (IMU, GPS, air-data) into a coherent state estimate. This project extends the fusion problem to include a remote-sensing channel (LiDAR) alongside the conventional state estimate, which is a comparatively less explored combination in the accessible literature and is a natural candidate for the project's original contribution.

2.6 Machine Learning in Aviation Safety

Machine learning has been applied to flight-data-monitoring anomaly detection, turbulence nowcasting from weather radar and satellite data, and predictive-maintenance applications. Tree-ensemble methods (random forests, gradient boosting) are attractive for tabular telemetry due to interpretability and low inference cost, while sequence models (LSTM, temporal convolutional networks) are better suited to capturing the time-dependent build-up of a gust encounter. Explainable AI (XAI) techniques are increasingly emphasized in safety-critical contexts, and are recommended in this project's evaluation plan (Chapter 10) rather than treated as optional.

2.7 Research Gap

The literature separately covers stall aerodynamics, AoA-based envelope protection, remote wind sensing, and ML-based flight anomaly detection — but a low-cost, prototype-scale system that integrates forward-looking LiDAR with AI-based stall-margin prediction, evaluated for lead-time advantage in simulation, has not been demonstrated at the capstone/research-prototype scale. This is the gap the proposed project addresses.

Note for the student: this chapter should be expanded with specific citations gathered from IEEE Xplore, NASA Technical Reports Server, and FAA Advisory Circulars during the literature search — the topics above indicate where to look; the actual citation list should be built from sources you have personally verified.

Chapter 3. System Architecture and Design

3.1 Concept of Operations

Forward-looking LiDAR units mounted near the wing leading edge or nose continuously scan the air ahead of the aircraft. Returns are converted into a coarse airflow disturbance estimate (a proxy for gust intensity and direction) and combined with the aircraft's inertial and air-data state. A machine learning model fuses these inputs to estimate the current and near-future effective angle of attack, from which stall margin and stall probability are derived and displayed to the pilot with a recommended corrective action.

3.2 High-Level Architecture

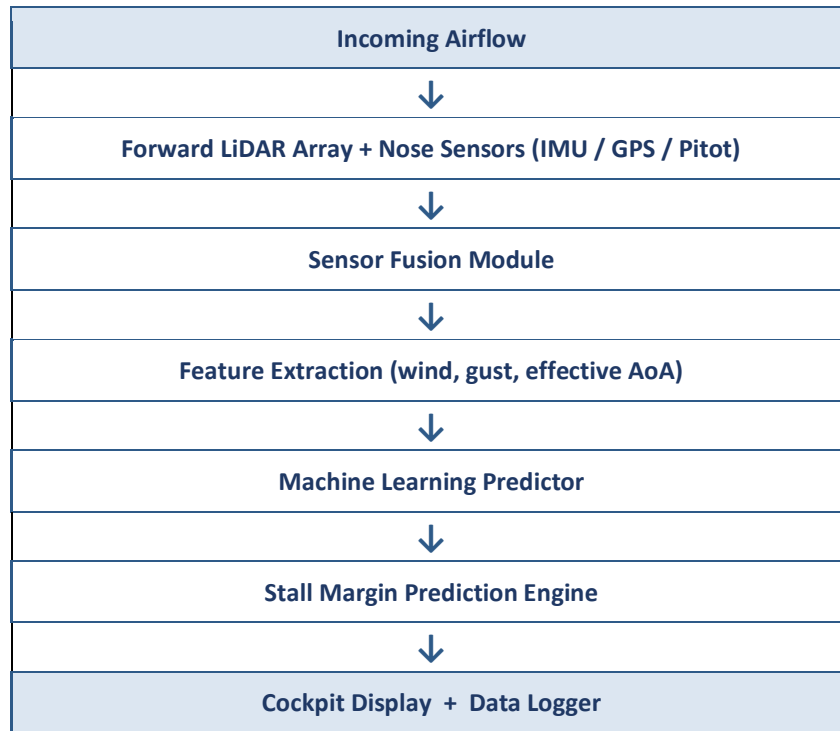


Figure 3.1 — Proposed end-to-end system architecture

3.3 Sensor Placement

Three LiDAR units are proposed on each side of the fuselage/wing root (L1–L3, R1–R3), oriented forward and slightly outboard to observe the approaching airflow over the left and right wing independently. This differential placement allows the system to detect asymmetric gust encounters (e.g., a gust affecting one wing before the other), which is relevant to roll upset in addition to pure stall margin. Nose-mounted IMU, GPS, and pitot-static sensors provide the aircraft's own state for fusion.

3.4 Data Flow

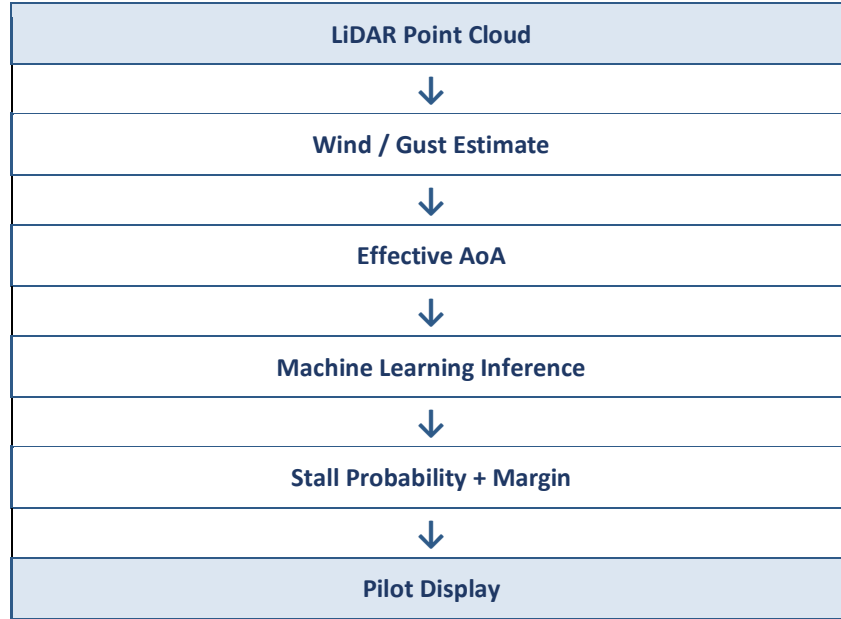
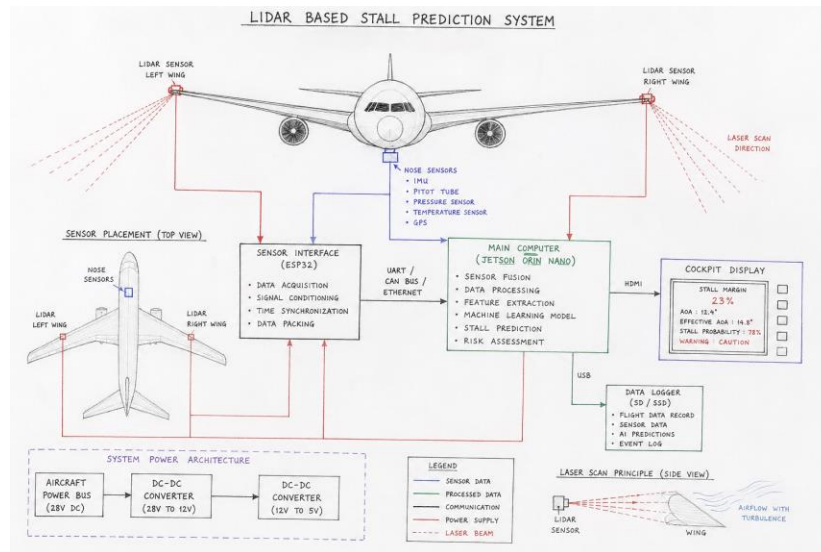


Figure 3.2 — Data flow from raw sensing to pilot-facing output

3.5 Communication and Interfaces

The embedded microcontroller (ESP32) polls the LiDAR and IMU over UART/I2C/SPI as appropriate to each device and packages readings over a serial or CAN link to the edge AI computer (Jetson Orin Nano), which performs sensor fusion, feature extraction, and model inference. Prediction results are pushed to the cockpit display over HDMI/USB and simultaneously logged to CSV/SQLite for later analysis. A production system would instead use an avionics-grade bus such as ARINC 429 or CAN Aerospace; this is noted as a future-work item rather than implemented in the prototype.



Chapter 4. Hardware Design

4.1 Component Selection Rationale

Two tiers of hardware are proposed: a low-cost tier suitable for an initial bench prototype and a professional tier for teams with additional budget or seeking closer-to-flight-representative fidelity. Both tiers are listed below with the reasoning for each choice.

Component	Prototype Option	Professional Option	Role
Forward LiDAR	Benewake TF03-100 (~\$180)	Livox Mid-360 (~\$600)	Forward airflow disturbance sensing
IMU	Bosch BNO055 (~\$35)	Xsens MTi-630 (~\$800)	Aircraft attitude (pitch/roll/yaw)
Air Data	Honeywell TruStability (~\$25)	Certified pitot-static probe	Backup airspeed / pressure reference
Embedded MCU	ESP32 DevKit (~\$10)	—	Sensor polling and packaging
Edge AI Compute	NVIDIA Jetson Orin Nano (~\$499)	—	Sensor fusion + model inference
Positioning	u-blox NEO-M8N GPS (~\$25)	—	Position / groundspeed reference
Display	7" HDMI touchscreen (~\$80)	Certified cockpit display	Pilot-facing stall margin readout

Table 4.1 — Bill of materials with prototype and professional-tier options (indicative pricing, subject to change)

4.2 Sensor Placement Diagram

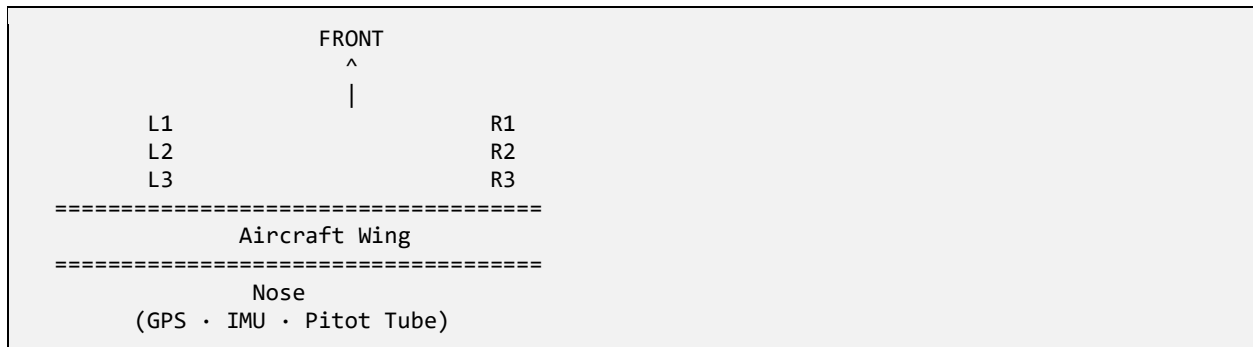


Figure 4.1 — Notional LiDAR placement for differential left/right airflow sensing

4.3 Power and Integration Considerations

The Jetson Orin Nano and multiple LiDAR units draw meaningfully more current than the ESP32 alone; a regulated 5V/12V power distribution board with adequate margin (rather than direct USB power) should be designed, along with a current budget for each rail. For a bench or simulator-tethered prototype, mains-derived bench supplies are acceptable; for any mobile or airframe-adjacent testing, battery capacity and thermal management of the Jetson module should be sized and verified before use.

4.4 Honest Limitations of the Sensing Approach

Time-of-flight and low-cost solid-state LiDAR units do not directly measure wind velocity; they measure range to aerosol/particulate returns and hard surfaces. Extracting a genuine wind vector requires either (a) a coherent Doppler LiDAR, which is substantially more expensive and complex than the prototype-tier hardware listed above, or (b) an indirect approach that infers gust effects from correlated changes in aircraft response and simulated environmental truth during training. This project should state clearly, in any presentation of results, which of these two approaches was actually used, since it materially affects how the results should be interpreted.

Chapter 5. Software Design

5.1 Software Architecture

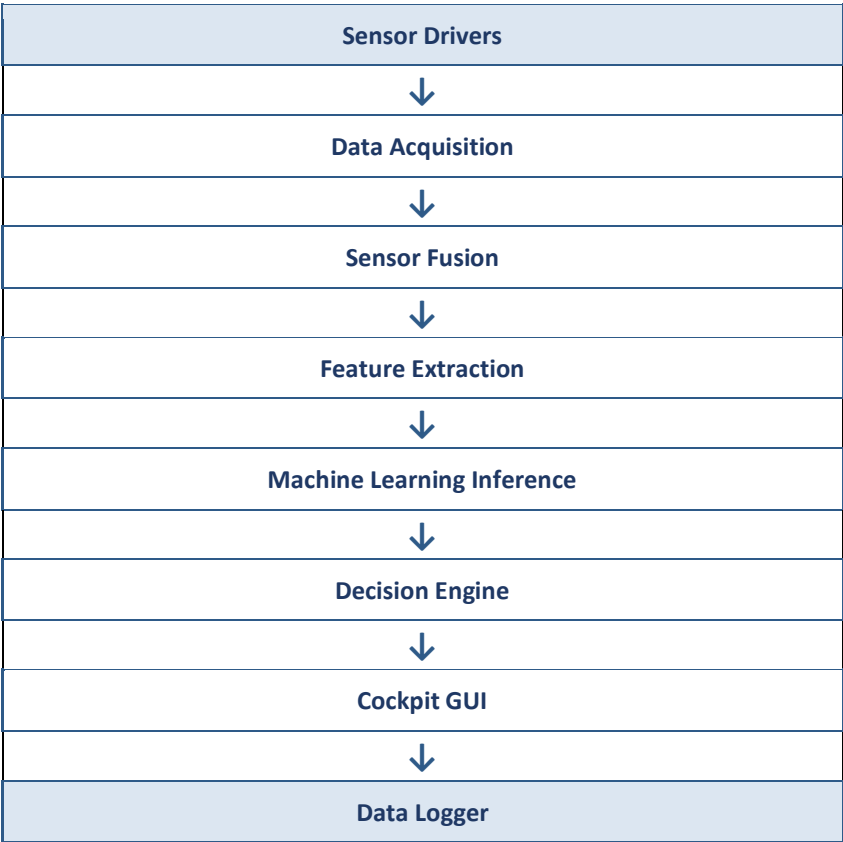


Figure 5.1 — Layered software architecture

5.2 Technology Stack

Layer	Technology
Application / AI	Python, scikit-learn, TensorFlow or PyTorch
Embedded firmware	C++ (ESP-IDF or Arduino framework)
Visualization	Streamlit or a lightweight custom GUI
Flight simulation	FlightGear (open source) or X-Plane
Data storage	CSV during development, SQLite for structured logging
LiDAR SDK	Manufacturer SDK (e.g., Livox SDK) or vendor UART protocol
Optional dashboard	Grafana, for time-series visualization of logged runs

Table 5.1 — Proposed software stack

5.3 Module Responsibilities

- Sensor Drivers — low-level read/parse routines per device, isolated behind a common interface so hardware can be swapped without touching downstream code.
- Sensor Fusion — time-synchronizes asynchronous sensor streams and produces a single fused state estimate (attitude, airspeed, estimated gust component).
- Feature Extraction — converts the fused state into the feature vector consumed by the ML model (see Chapter 7).
- Decision Engine — applies thresholds/logic on top of the model output to generate a discrete alert level and recommended action.
- Cockpit GUI — renders current AoA, effective AoA, stall margin, stall probability, and recommended action in a glanceable format.
- Data Logger — records every run to CSV/SQLite for offline analysis and for building the training dataset.

Chapter 6. Mathematical Framework

6.1 Lift and Drag

$$L = 1/2 \rho V^2 S C_L$$

$$D = 1/2 \rho V^2 S C_D$$

where ρ is air density, V is airspeed, S is wing reference area, and C_L / C_D are the lift and drag coefficients, which are themselves functions of angle of attack.

6.2 Effective Angle of Attack Under Gust

$$AoA_{eff} = AoA_{aircraft} + \Delta AoA_{gust}$$

6.3 Stall Margin

$$SM = AoA_{critical} - AoA_{eff}$$

Stall margin (SM) is the core quantity the system predicts. A positive margin indicates safe operation; a margin approaching zero indicates imminent stall risk. The classification thresholds used in the reference implementation (Chapter 8) are illustrative and should be re-derived from the specific aircraft's real stall characteristics if the project is extended beyond a generic simulated airframe.

6.4 Stall Probability as a Learned Function

$$P(stall) = f(AoA, V, gust, pitch, roll; \theta)$$

where f is the trained machine learning model parameterized by θ (Chapter 7). Framing stall risk as a learned probability, rather than a hard threshold, allows the system to express graded risk levels (Safe / Caution / High Risk / Stall Warning) instead of a single binary alert.

6.5 Evaluation Metrics

Model quality is assessed using standard classification metrics — precision, recall, F1 score, and the ROC curve / area under the curve — computed on a held-out simulation test set, in addition to a domain-specific lead-time metric (Chapter 10) that measures how many seconds before actual stall onset the system raises a correct warning.

Chapter 7. Machine Learning Methodology

7.1 Problem Framing

The core prediction task is framed as a multi-class classification problem (Safe / Near-Stall / Stall) with an accompanying regression output for stall margin in degrees, so that both a discrete alert and a continuous margin value are available to the decision engine and cockpit display.

7.2 Candidate Features

Category	Features
Aircraft state	Angle of attack, pitch, roll, yaw, airspeed, vertical speed, altitude, flap setting, weight, engine thrust
Environmental	Estimated wind speed / direction, gust magnitude, humidity, temperature
LiDAR-derived	Point-cloud-derived disturbance intensity, left/right asymmetry, rate of change

Table 7.1 — Candidate feature set

7.3 Target Labels

Labels (Safe, Near-Stall, Stall) are generated from the simulator's ground-truth aerodynamic state at each timestep, offset forward in time so the model learns to predict the label a few seconds ahead of the labeled event rather than merely classifying the current instant — this forward offset is what gives the system its predictive, rather than purely reactive, character.

7.4 Candidate Models

- Random Forest / Gradient Boosted Trees — strong baseline for tabular telemetry, fast to train, reasonably interpretable via feature importance.
- Feed-forward Neural Network — captures nonlinear interactions between features; requires more data and tuning than tree ensembles.
- LSTM / Temporal Convolutional Network (stretch goal) — models the time history of a gust encounter rather than a single instant, which may improve lead-time performance at the cost of implementation complexity.

7.5 Training and Validation Protocol

Data should be split by simulation run (not by individual timestep) into training, validation, and held-out test sets to avoid leakage between temporally adjacent samples of the same event. Cross-validation across multiple simulated turbulence intensities and aircraft configurations is recommended to assess generalization rather than relying on a single train/test split.

7.6 Explainability

Given the safety-adjacent framing of this project, feature-importance analysis (e.g., permutation importance or SHAP values) is recommended as a standard part of the evaluation, both to build confidence in the model's reasoning and to identify whether the LiDAR-derived features are contributing meaningfully beyond conventional telemetry alone — a key question for justifying the added sensing hardware.

Chapter 8. Reference Implementation

8.1 Deterministic Baseline (Rule-Based)

Before training a model, a simple deterministic baseline is useful both as a sanity check and as a fallback mode. The example below computes stall margin directly from the equations in Chapter 6:

```
import math

def stall_margin(aoa_deg, gust_deg, critical_aoa_deg=16.0):
    effective_aoa = aoa_deg + gust_deg
    margin = critical_aoa_deg - effective_aoa
    if margin <= 0:
        status = "STALL WARNING"
    elif margin < 2:
        status = "HIGH RISK"
    elif margin < 4:
        status = "CAUTION"
    else:
        status = "SAFE"
    return effective_aoa, margin, status

eff, margin, status = stall_margin(aoa_deg=13, gust_deg=2.5)
print(f"{status}: effective AoA={eff:.1f} deg, margin={margin:.1f} deg")
```

Listing 8.1 — Deterministic stall-margin baseline

8.2 Illustrative Machine Learning Prototype

```
from sklearn.ensemble import RandomForestClassifier
import numpy as np

# Columns: [angle_of_attack_deg, gust_deg, airspeed_ms]
X_train = np.array([
    [12, 1.2, 78],
    [14, 2.0, 72],
    [15, 3.0, 69],
    [10, 0.5, 90],
])
y_train = np.array([0, 1, 1, 0]) # 0 = Safe, 1 = Stall risk

model = RandomForestClassifier(n_estimators=200, random_state=42)
model.fit(X_train, y_train)

sample = np.array([[13, 2.5, 74]])
print(model.predict(sample), model.predict_proba(sample))
```

Listing 8.2 — Minimal illustrative classifier (replace with simulator-derived dataset before drawing conclusions)

Both listings above are illustrative starting points with a handful of hand-picked rows; they are not a substitute for the simulator-generated dataset described in Chapter 9, and no performance conclusions should be drawn from them.

8.3 Software Modules to Implement

- LidarInterface — reads and parses the selected LiDAR's native protocol into a common point-cloud / range structure.
- SensorFusion — time-aligns and fuses IMU, GPS, air-data, and LiDAR-derived features into one state vector per timestep.
- StallPredictor — wraps the trained model (rule-based fallback + ML model) behind a single predict(state) interface.
- CockpitDisplay — Streamlit or custom GUI rendering current/effective AoA, margin, probability, and recommended action.
- DataLogger — appends every timestep's inputs and outputs to CSV/SQLite for later analysis and retraining.

Chapter 9. Simulation and Test Plan

9.1 Simulation Environment

FlightGear is recommended as the primary simulation environment for its open-source flight dynamics model, scriptable environment (via its property tree and FGPython/FlightGear-python bridges), and zero licensing cost; X-Plane is a viable alternative with a more sophisticated blade-element aerodynamic model, at the cost of a commercial license. Either can expose ground-truth AoA, airspeed, and turbulence state needed to label the training dataset.

9.2 Test Scenarios

Scenario	Purpose
Steady level flight	Baseline / negative-control data (no stall risk expected)
Crosswind approach	Asymmetric gust response, left/right LiDAR differential
Wind shear on approach	Rapid AoA excursion under classic accident-relevant conditions
Mountain-wave / rotor turbulence	Sustained turbulence, tests robustness over longer encounters
Wake-vortex encounter	Short, sharp disturbance -- tests system latency
Full-stall approach to landing	High-AoA, low-speed regime where false negatives are most costly
Takeoff with gusting crosswind	Low-altitude, high-consequence scenario

Table 9.1 — Proposed simulator test matrix

9.3 Metrics Collected Per Run

- Ground-truth AoA, stall margin, and stall onset time (from the simulator).
- Model-predicted stall margin, stall probability, and alert level at each timestep.
- Lead time: seconds between the model's first correct high-risk alert and actual stall onset.
- False-positive rate: fraction of safe-flight time incorrectly flagged as high risk.

9.4 Hardware-in-the-Loop (Optional Stretch Goal)

If institutional safety review and budget allow, the physical LiDAR/IMU rig can be mounted on a ground vehicle or a wind-tunnel fixture to validate that real sensor noise characteristics behave as assumed in the simulated dataset, before drawing conclusions that depend on real hardware. This should be treated as an optional extension, not a requirement for a successful capstone.

Chapter 10. Expected Results and Evaluation

10.1 Success Criteria

- The system produces a correct high-risk classification with a measurable positive lead time (target: at least 2-3 seconds, to be refined once baseline data is collected) ahead of simulated stall onset in the majority of gust/turbulence test scenarios.
- False-positive rate during steady, safe flight remains low enough that the alert would not be dismissed as noise by a pilot (a specific numeric target should be set once baseline data is available).
- Ablation study shows a measurable difference in performance between models using LiDAR-derived features and models using aircraft telemetry alone -- this is the key evidence for whether the added LiDAR hardware is justified.

10.2 Reporting Format

Results should be reported with confusion matrices, ROC curves, and precision/recall for each test scenario individually as well as pooled, alongside the lead-time distribution (e.g., as a histogram) and a clear statement of the ablation comparison described above. All results should be reported honestly, including scenarios where the system underperforms, since a capstone thesis is evaluated on the rigor of the evaluation as much as on the headline numbers.

10.3 Threats to Validity

- Simulator fidelity: FlightGear/X-Plane aerodynamic models are approximations; results should be described as simulation-validated, not flight-validated.
- Synthetic LiDAR data: if physical hardware-in-the-loop testing is not performed, LiDAR-derived features may need to be synthesized from the simulator's wind field, which should be disclosed explicitly rather than implied to be real sensor data.
- Label leakage: care must be taken that forward-shifted labels do not inadvertently allow the model to see post-event information during training.

Chapter 11. Future Work

- Doppler LiDAR integration for a genuine, direct wind-vector measurement rather than an indirect proxy.
- Fusion with weather radar and/or satellite turbulence nowcasting for longer-horizon risk awareness.
- Sequence models (LSTM / Transformer) trained on longer time histories of the gust encounter.
- Reinforcement-learning-based recommended-action policies, rather than static thresholds.
- A 'digital twin' of a specific airframe for higher-fidelity, aircraft-specific stall modeling.
- Augmented-reality cockpit presentation of stall margin and recommended recovery attitude.
- A structured review of DO-178C (software) and DO-254 (hardware) considerations, and of ARINC 429 / CAN Aerospace integration, as a roadmap toward eventual certification -- explicitly out of scope for this capstone but valuable to document as next steps.

Chapter 12. Conclusion

This proposal defines a scoped, achievable capstone project that investigates whether forward-looking LiDAR, fused with conventional flight telemetry and analyzed with machine learning, can provide a measurable lead-time advantage over today's reactive stall-warning systems. By integrating advanced sensing technologies with artificial intelligence, the proposed system aims to estimate the aircraft's remaining stall margin before hazardous flight conditions develop, enabling earlier pilot awareness and more informed decision-making. The project will be validated through simulation-based testing using representative turbulence scenarios, allowing the predictive model to be evaluated safely and cost-effectively. While the proposed system is intended as a research prototype rather than a certified aviation solution, its findings could contribute to future developments in intelligent flight safety systems, predictive avionics, and AI-assisted flight envelope protection. Ultimately, this research demonstrates how modern sensor fusion and machine learning techniques can be applied to enhance aviation safety by shifting stall detection from a reactive warning mechanism to a proactive prediction system.